

Module 1

Lab - Write Your First Python Code

Say Hello to the world in Python

Code:

```
print('Hello, Python!')
```

Output:

```
Hello, Python!
```

What version of Python are we using?

Code:

```
import sys
print(sys.version)
```

Output:

```
3.11.3 (main, ...) [this line depends on your own lab environment]
```

Writing comments in Python

Code:

```
print('Hello, Python!') # This line prints a string
# print('Hi')
```

Output:

```
Hello, Python!
```

Errors in Python

Code:

```
frint("Hello, Python!")
```

Output:

```
NameError: name 'frint' is not defined
```

Code:

```
print("Hello, Python!")
```

Output:

```
SyntaxError: unterminated string literal (detected at line 1)
```

Does Python know about your error before it runs your code?

Code:

```
print("This will be printed")
frint("This will cause an error")
```

```
print("This will NOT be printed")
```

Output:

This will be printed

NameError: name 'frint' is not defined

Exercise: Your First Program

Code:

```
print('Hello, world!')
```

Output:

Hello, world!

Code:

```
print('Hello, world!') # Print the traditional hello world
```

Output:

Hello, world!

Types of objects in Python

Code:

```
type(12)
```

Output:

<class 'int'>

Code:

```
type(2.14)
```

Output:

<class 'float'>

Code:

```
type("Hello, Python 101!")
```

Output:

<class 'str'>

Code:

```
type(12.0)
```

Output:

<class 'float'>

Integers

Code:

```
type(-1)
```

Output:

```
<class 'int'>
```

Code:
type(4)

Output:
<class 'int'>

Code:
type(0)

Output:
<class 'int'>

Floats

Code:
type(1.0)

Output:
<class 'float'>

Code:
type(0.5)

Output:
<class 'float'>

Code:
type(0.56)

Output:
<class 'float'>

Code:
sys.float_info

Output:
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308,
min=2.2250738585072014e-308, min_exp=-1021, min_10_exp=-307, dig=15,
mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)

Converting from one object type to a different object type

Code:
type(2)

Output:
<class 'int'>

Converting integers to floats

Code:

```
float(2)
```

Output:

```
2.0
```

Code:

```
type(float(2))
```

Output:

```
<class 'float'>
```

Code:

```
int(1.1)
```

Output:

```
1
```

Converting from strings to integers or floats

Code:

```
int('1')
```

Output:

```
1
```

Code:

```
int('1 or 2 people')
```

Output:

```
ValueError: invalid literal for int() with base 10: '1 or 2 people'
```

Code:

```
float('1.2')
```

Output:

```
1.2
```

Converting numbers to strings

Code:

```
str(1)
```

Output:

```
'1'
```

Code:

```
str(1.2)
```

Output:

```
'1.2'
```

Boolean data type

Code:

True

Output:

True

Code:

False

Output:

False

Code:

type(True)

Output:

<class 'bool'>

Code:

type(False)

Output:

<class 'bool'>

Code:

int(True)

Output:

1

Code:

bool(1)

Output:

True

Code:

bool(0)

Output:

False

Code:

float(True)

Output:

1.0

Exercise: Types

Code:

type(6 / 2)

Output:
<class 'float'>

Code:
type(6 // 2)

Output:
<class 'int'>

Expressions and Variables

Code:
43 + 60 + 16 + 41

Output:
160

Code:
50 - 60

Output:
-10

Code:
5 * 5

Output:
25

Code:
25 / 5

Output:
5.0

Code:
25 / 6

Output:
4.166666666666667

Code:
25 // 5

Output:
5

Code:
25 // 6

Output:
4

Exercise: Expressions

Code:

```
160 / 60
```

Output:

```
2.6666666666666665
```

Code:

```
30 + 2 * 60
```

Output:

```
150
```

Code:

```
(30 + 2) * 60
```

Output:

```
1920
```

Variables

Code:

```
x = 43 + 60 + 16 + 41
```

Output:

```
no output, this line just stores the value
```

Code:

```
x
```

Output:

```
160
```

Code:

```
y = x / 60  
y
```

Output:

```
2.6666666666666665
```

Code:

```
x = x / 60  
x
```

Output:

```
2.6666666666666665
```

Code:

```
total_min = 43 + 42 + 57 # Total length of albums in minutes  
total_min
```

Output:

142

Code:

```
total_hours = total_min / 60 # Total length of albums in hours
total_hours
```

Output:

2.3666666666666667

Code:

```
total_hours = (43 + 42 + 57) / 60
total_hours
```

Output:

2.3666666666666667

Exercise: Expression and Variables in Python

Code:

```
x = 3 + 2 * 2
x
```

Output:

7

Code:

```
y = (3 + 2) * 2
y
```

Output:

10

Code:

```
z = x + y
z
```

Output:

17

Module 1 completed.

Lab - String Operations

What are Strings?

Code:

```
"The BodyGuard"
```

Output:

```
'The BodyGuard'
```


Code:
'The BodyGuard'

Output:
'The BodyGuard'

Code:
'1 2 3 4 5 6 '

Output:
'1 2 3 4 5 6 '

Code:
'@#2_#]&*^%\$'

Output:
'@#2_#]&*^%\$'

Code:
print("hello!")

Output:
hello!

Code:
name = "The BodyGuard"
name

Output:
'The BodyGuard'

Indexing

Code:
print(name[0])

Output:
T

Code:
print(name[6])

Output:
d

Code:
print(name[10])

Output:
a

Negative Indexing

Code:
`print(name[-1])`

Output:
d

Code:
`print(name[-13])`

Output:
T

Code:
`len("The BodyGuard")`

Output:
13

Slicing

Code:
`name[0:4]`

Output:
'The '

Code:
`name[8:12]`

Output:
'Guar'

Stride

Code:
`name[::2]`

Output:
'TeBdGad'

Code:
`name[0:5:2]`

Output:
'TeB'

Concatenate Strings

Code:
`statement = name + " is the best album"`
`statement`

Output:

'The BodyGuard is the best album'

Code:

```
3 * "The BodyGuard"
```

Output:

'The BodyGuardThe BodyGuardThe BodyGuard'

Code:

```
name = "The BodyGuard"  
name = name + " is the best album"  
name
```

Output:

'The BodyGuard is the best album'

Escape Sequences

Code:

```
print(" The BodyGuard\n is the best album" )
```

Output:

```
The BodyGuard  
is the best album
```

Code:

```
print(" The BodyGuard\t is the best album" )
```

Output:

```
The BodyGuard      is the best album
```

Code:

```
print(" The BodyGuard \\ is the best album" )
```

Output:

```
The BodyGuard \ is the best album
```

Code:

```
print(r" The BodyGuard \ is the best album" )
```

Output:

```
The BodyGuard \ is the best album
```

String Manipulation Operations

Code:

```
a = "Thriller is the sixth studio album"  
print("before upper:", a)  
b = a.upper()  
print("After upper:", b)
```

Output:

before upper: Thriller is the sixth studio album
After upper: THRILLER IS THE SIXTH STUDIO ALBUM

Code:

```
a = "The BodyGuard is the best album"  
b = a.replace('BodyGuard', 'Janet')  
b
```

Output:

```
'The Janet is the best album'
```

Code:

```
name = "The BodyGuard"  
name.find('he')
```

Output:

```
1
```

Code:

```
name.find('Guard')
```

Output:

```
8
```

Code:

```
name.find('Jasdfasdf')
```

Output:

```
-1
```

Code:

```
name = "The BodyGuard"  
split_string = (name.split())  
split_string
```

Output:

```
['The', 'BodyGuard']
```

RegEx

Code:

```
import re
```

Output:

no output, this line just loads the module

Code:

```
s1 = "The BodyGuard is the best album"  
  
pattern = r"Body"  
  
result = re.search(pattern, s1)  
  
if result:
```

```
    print("Match found!")
else:
    print("Match not found.")
```

Output:

Match found!

Code:

```
pattern = r"\d\d\d\d\d\d\d\d\d\d"
text = "My Phone number is 1234567890"
match = re.search(pattern, text)

if match:
    print("Phone number found:", match.group())
else:
    print("No match")
```

Output:

Phone number found: 1234567890

Code:

```
pattern = r"\W"
text = "Hello, world!"
matches = re.findall(pattern, text)

print("Matches:", matches)
```

Output:

Matches: [',', ' ', '!', '']

Code:

```
s2 = "The BodyGuard is the best album of 'Whitney Houston'."

result = re.findall("st", s2)

print(result)
```

Output:

['st', 'st']

Code:

```
split_array = re.split(r"\s", s2)

print(split_array)
```

Output:

['The', 'BodyGuard', 'is', 'the', 'best', 'album', 'of', "'Whitney",
"Houston'."]

Code:

```
pattern = r"Whitney Houston"
replacement = "legend"
new_string = re.sub(pattern, replacement, s2, flags=re.IGNORECASE)

print(new_string)
```

Output:

The BodyGuard is the best album of 'legend'.

Quiz on Strings**Code:**

```
a = "1"
```

Output:

no output, this line just stores the value

Code:

```
b = "2"
```

Output:

no output, this line just stores the value

Code:

```
c = a + b  
c
```

Output:

```
'12'
```

Code:

```
d = "ABCDEFGH"  
print(d[:3])  
# or  
print(d[0:3])
```

Output:

```
ABC  
ABC
```

Code:

```
e = 'clocrkrlelclt'  
print(e[::2])
```

Output:

```
correct
```

Code:

```
print("\\")
```

Output:

```
\
```

Code:

```
f = "You are wrong"  
f.upper()
```

Output:

```
'YOU ARE WRONG'
```

Code:

```
f2 = "YOU ARE RIGHT"  
f2.lower()
```

Output:

```
'you are right'
```

Code:

```
g = "Mary had a little lamb Little lamb, little lamb Mary had a little lamb  
Its fleece was white as snow And everywhere that Mary went Mary went, Mary  
went Everywhere that Mary went The lamb was sure to go"  
g.find('snow')
```

Output:

```
95
```

(note: this number can shift by a character or two depending on the exact spacing you typed in your multi-line string)

Code:

```
g.replace('Mary', 'Bob')
```

Output:

```
'Bob had a little lamb Little lamb, little lamb Bob had a little lamb Its  
fleece was white as snow And everywhere that Bob went Bob went, Bob went  
Everywhere that Bob went The lamb was sure to go'
```

Code:

```
g.replace(',', ' .')
```

Output:

```
'Mary had a little lamb Little lamb. little lamb Mary had a little lamb Its  
fleece was white as snow And everywhere that Mary went Mary went. Mary went  
Everywhere that Mary went The lamb was sure to go'
```

Code:

```
g.split()
```

Output:

```
['Mary', 'had', 'a', 'little', 'lamb', 'Little', 'lamb,', 'little', 'lamb',  
'Mary', 'had', 'a', 'little', 'lamb', 'Its', 'fleece', 'was', 'white', 'as',  
'snow', 'And', 'everywhere', 'that', 'Mary', 'went', 'Mary', 'went,', 'Mary',  
'went', 'Everywhere', 'that', 'Mary', 'went', 'The', 'lamb', 'was', 'sure',  
'to', 'go']
```

Code:

```
s3 = "House number- 1105"  
result = re.search(r'\d', s3)  
result
```

Output:

```
<re.Match object; span=(14, 15), match='1'>
```

Code:

```
str1 = "The quick brown fox jumps over the lazy dog."
```

```
re.sub(r'fox', 'bear', str1)
```

Output:

```
'The quick brown bear jumps over the lazy dog.'
```

Code:

```
str2 = "How much wood would a woodchuck chuck, if a woodchuck could chuck  
wood?"  
matches = re.findall(r"woo", str2)  
print(matches)
```

Output:

```
['woo', 'woo', 'woo', 'woo']
```

Module completed - Strings.